LLM

↳ Pre-trained on huge amount internet data till a cut-off data.

Hallucination

LLM + Knowledge Source

RAG

⇒ Retreival Augmented Generation

Model generates a response (augmented or enhanced) by a retreiving relevant information from internal database.

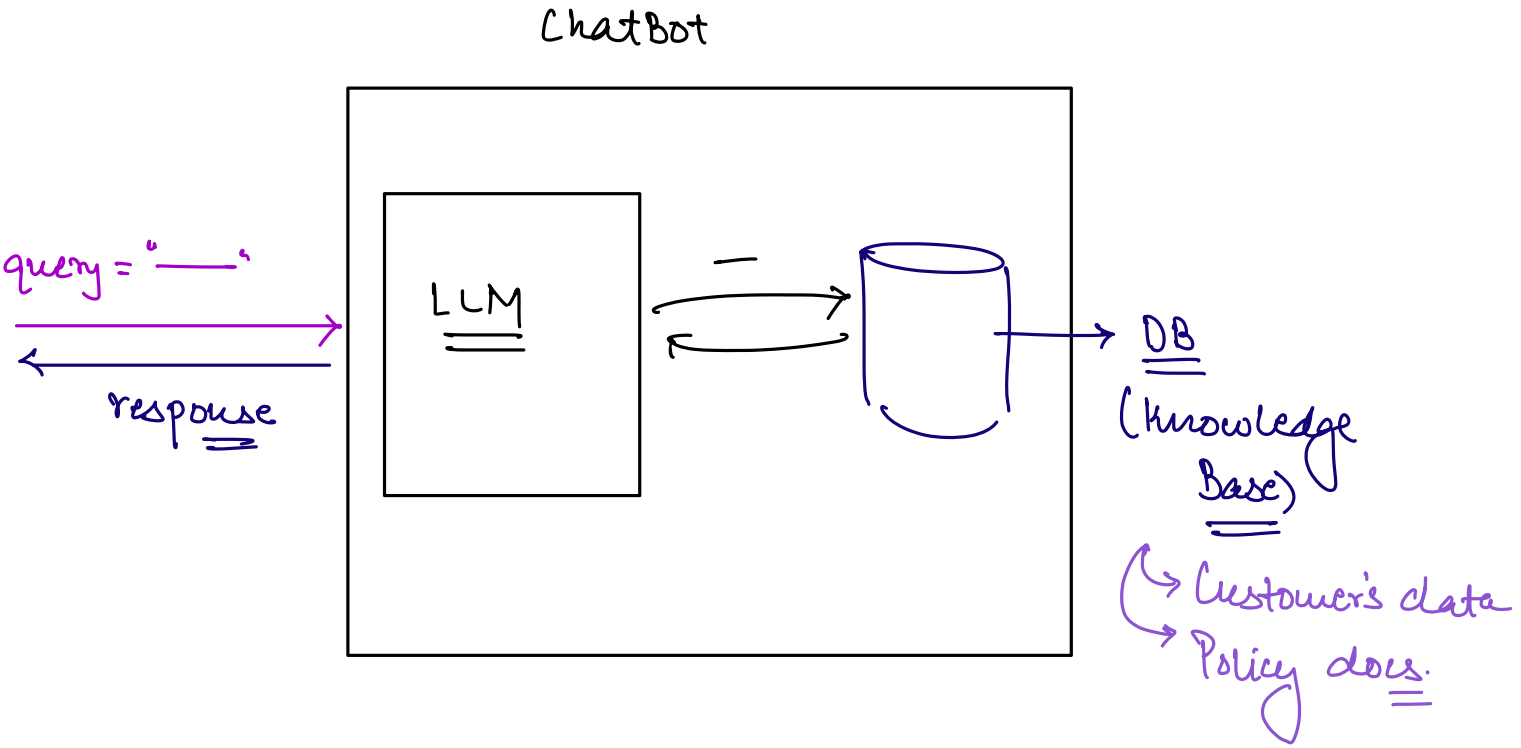
Retreival + Generation

↓ Relevant information from DB

↳ Generate the final response using information fetched from DB.

RAG is a technique in AI Applications that improves the LLM by fetching the facts from the knowledge source. This stops LLM from guessing or making-up the answers.

How RAG works.



→ **Ingestion**: Policy documents or files are divided into small chunks and saved in a special type of DB which is VECTOR DB.

→ **Retrieval** : When the user query comes, the system searches the relevant chunk from the **Vector DB**.

→ **Generation** : The model generates the final response using the retrieved info.

Phase-1 : **INDEXING**

⇒ Take the documents + Split into Chunks + Generate Embeddings for each Chunk + Store these Embeddings into Vector DB.

Phase-2 : **QUERY**

⇒ User asks a query + Generate Embeddings for query + Perform Similarity Search & get the relevant Chunks + Pass these relevant Chunks to the LLM & generate the final response.

RAG Demo

```
User question: "What is our refund policy for annual subscriptions?"

Step 1 — Embed the question
  question_embedding = generate_embedding(user_question)

Step 2 — Search the indexed knowledge base
  relevant_chunks = vector_search(question_embedding, top_k=3)
  → Returns the 3 most similar stored chunks, e.g.:
    - "Annual subscriptions are refundable within 30 days..."
    - "Refund requests must be submitted through the billing portal..."
    - "Partial refunds are not offered after the 30-day window..."

Step 3 — Build an augmented prompt
  prompt = f"""
  Answer the question using only the following context.

  Context:
  {relevant_chunks}

  Question: {user_question}
  """

Step 4 — Generate the answer
  response = call_llm(prompt)
  → "Annual subscriptions can be refunded within 30 days of
     purchase. Requests must be submitted through the billing
     portal, and partial refunds are not available after that
     30-day window."
```

Vector Search

→ In Vector DB, data is stored in the form of embeddings.

⇒ To search in Vector DB, query is converted into embedding & then similarity search is performed to retrieve the most similar documents to the query.

Chunking Documents before Embeddings

→ Documents can be very lengthy so Instead of creating the embedding of the entire document, we first divide the document into Chunks and then create embeddings of these Chunks.

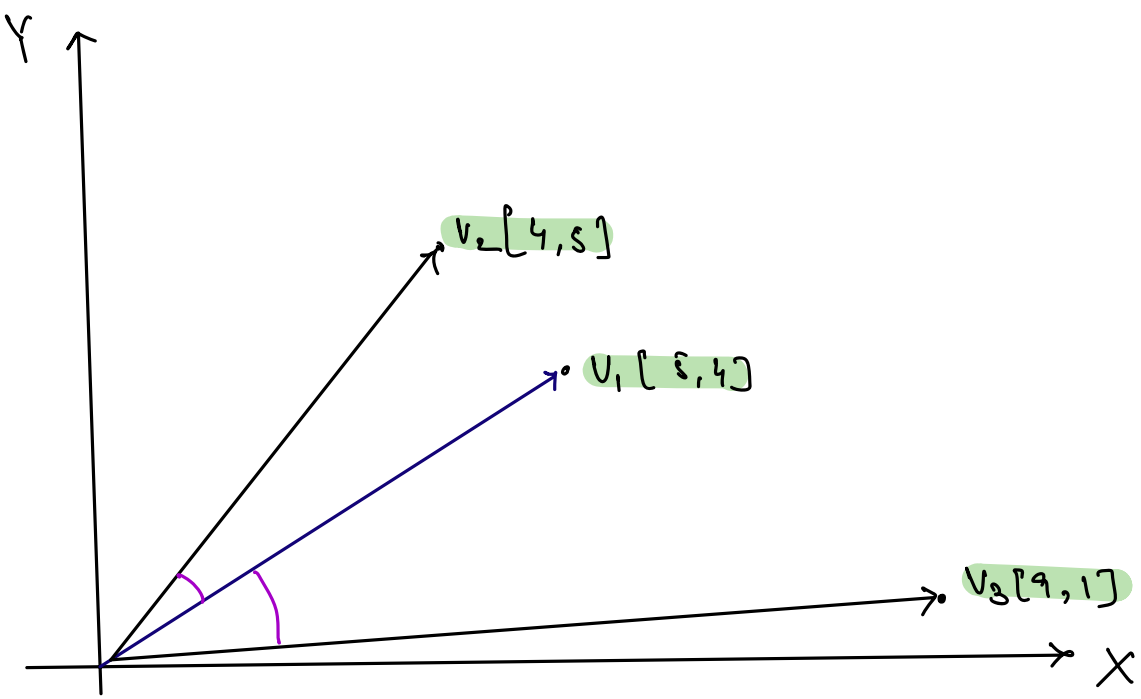
Chunking makes storing & retrieving data from Vector DB more efficient.

```
Original document (long):
"Section 1: Company Overview... [500 words]
 Section 2: Refund Policy... [300 words]
 Section 3: Shipping Policy... [400 words]
 Section 4: Contact Information... [100 words]"

After chunking (each chunk embedded separately):
Chunk 1: "Section 1: Company Overview..." → embedding A
Chunk 2: "Section 2: Refund Policy..."    → embedding B
Chunk 3: "Section 3: Shipping Policy..."  → embedding C
Chunk 4: "Section 4: Contact Information" → embedding D

A question about refunds should match closely with embedding B
specifically, not the whole document as one blurred-together vector.
```

⇒ A vector database is a specialized data store built specifically for fast similarity searches across large no. of vector embeddings.



Similar Vectors ⇒ Closer to each other.

Distance b/w the Vectors ↑ ⇒ Similarity ↓

$$\text{Similarity} \propto \frac{1}{\text{Distance}}$$

Grounding

"Grounding means constraining the model's generated response to be based on specific, provided information, rather than letting it rely purely on whatever patterns it learned during training. This is the actual mechanism by which RAG reduces hallucination — not by making the model inherently smarter or more careful, but by giving it real source material to work from and instructing it to use that material specifically.

The retrieval phase finds the relevant chunks. Grounding is what happens next: how those chunks are actually used to shape the generated answer, through prompt structure and explicit instruction."

Prompts for Grounded Generation

```
UNGROUNDED PROMPT (retrieval happened, but grounding wasn't enforced):
"Here is some context: {retrieved_chunks}
 Answer this question: {user_question}"

Risk: the model may blend the provided context with its own
      training-data assumptions, or answer confidently even if
      the context does not actually contain the answer.
```

```
GROUNDED PROMPT:
"Answer the question using ONLY the information in the context
below. If the context does not contain enough information to
answer the question, say so explicitly rather than guessing.

Context:
{retrieved_chunks}

Question: {user_question}"
```

RAG Accuracy

RAG improves accuracy in two distinct, measurable ways. First, it gives the model access to information it was never trained on at all — private company data, real-time information, anything created after the model's training cutoff. Second, even for information the model might technically know from training, grounding it in an authoritative retrieved source reduces the chance of it confidently stating a subtly wrong version of that information from memory."

Summary

- RAG combines retrieval (search) with generation (text production), grounding the model's output in retrieved information rather than relying solely on training data
- Indexing happens ahead of time — chunking, embedding, and storing documents; query time happens live for every user question — embedding the question, retrieving relevant chunks, and generating a grounded answer
- RAG does not retrain or fine-tune the model; it changes what information is available in the prompt for each specific request

Vector Search

- Vector search applies the embeddings and cosine similarity technique from the embeddings session to finding relevant chunks at scale
- Documents are chunked before embedding, since a single vector cannot usefully represent an entire lengthy document; chunk size is a real tradeoff between context loss and topic blur
- A vector database stores embeddings with their original text and enables fast similarity search across large collections without a slow one-by-one comparison loop

Knowledge Grounding

- Grounding constrains generation to rely on retrieved information rather than the model's own training-data patterns
- Simply including retrieved context is not enough — the prompt must explicitly instruct the model to rely on it, with an explicit fallback for when the context is insufficient
- An honest "the available information does not cover this" is the correct outcome of a well-grounded system, not a failure of it